

# **EMERGENCY VEHICLE TRAFFIC DETECTION USING YOLO**

## **A PROJECT REPORT**

*Submitted by*

**DIVYANSH AGARWAL (25BCE11367)**

**VISHAL SONI (25BCE11434)**

**SRIKANT KHATRI (25BCE10641)**

**RITAM DEEP CHOWDHURY (25BCE11317)**

**PRANAV CHAUHAN (25BCE11289)**

*in partial fulfillment for the award of the degree  
of*

**BACHELOR OF TECHNOLOGY**

*in*

**COMPUTER SCIENCE AND ENGINEERING**



**SCHOOL OF COMPUTING SCIENCE AND ENGINEERING**

**VIT BHOPAL UNIVERSITY**

**KOTRIKALAN, SEHORE**

**MADHYA PRADESH - 466114**

October 2026

**VIT BHOPAL UNIVERSITY, KOTHRIKALAN,  
SEHORE**

**MADHYA PRADESH – 466114**

**BONAFIDE CERTIFICATE**

Certified that this project report titled “EMERGENCY VEHICLE TRAFFIC DETECTION USING YOLO” is the bonafide work of “Divyansh Agarwal (25BCE11367), Vishal Soni (25BCE11434), Pranav Chauhan (25BCE11289), Srikant Khatri (25BCE10641) and Ritam Deep Chowdhury (25BCE11317)” who carried out the project work under my supervision. Certified further that to the best of my knowledge the work reported at this time does not form part of any other project/research work based on which a degree or award was conferred on an earlier occasion on this or any other candidate.

|                                                                      |                                                                                                                    |
|----------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| PROGRAM CHAIR- DR.Vikas Panthi                                       | PROJECT GUIDE                                                                                                      |
| School of Computing Science and Engineering<br>VIT BHOPAL UNIVERSITY | Ms. Manorma Chouhan<br>Assistant Professor<br>School of Computing Science and Engineering<br>VIT BHOPAL UNIVERSITY |

The Project Exhibition I 09/10/2026

## **ACKNOWLEDGEMENT**

First and foremost, we would like to thank the Lord Almighty for His presence and immense blessings throughout the project work.

We wish to express our heartfelt gratitude to Dr. D. Lakshmi, Head of the Department, School of Computing Science and Engineering, for her valuable support, strategic direction, and encouragement in successfully carrying out this work.

We would like to thank our internal guide Ms. Manorma Chouhan, for continually guiding and actively participating in our project, and for giving valuable suggestions to complete the project work.

We would like to thank all the technical and teaching staff of the School of Computing Science and Engineering, who extended directly or indirectly all support.

Last, but not least, we are deeply indebted to our parents, who have been the greatest support while we worked day and night to make the project a success.

Divyansh Agarwal

Vishal Soni

Pranav Chauhan

Srikant Khatri

Ritam Deep Chowdhury

## LIST OF ABBREVIATIONS

| Abbreviation | Full Form                                 |
|--------------|-------------------------------------------|
| AI           | Artificial Intelligence                   |
| CCTV         | Closed-Circuit Television                 |
| CNN          | Convolutional Neural Network              |
| CPU          | Central Processing Unit                   |
| CUDA         | Compute Unified Device Architecture       |
| FPS          | Frames Per Second                         |
| GPU          | Graphics Processing Unit                  |
| HOG          | Histogram of Oriented Gradients           |
| IoU          | Intersection over Union                   |
| mAP          | Mean Average Precision                    |
| R-CNN        | Region-based Convolutional Neural Network |
| SSD          | Single Shot MultiBox Detector             |
| SVM          | Support Vector Machine                    |
| UI           | User Interface                            |
| YOLO         | You Only Look Once                        |

## LIST OF FIGURES AND GRAPHS

| Figure No. | Title                                                                   | Page No. |
|------------|-------------------------------------------------------------------------|----------|
| 2.1        | Detailed YOLOv11 neural network architecture (Backbone, Neck, Head)     | 6        |
| 3.1        | System Use Case Requirement Specification Artifact                      | 10       |
| 3.2        | Level-1 Data Flow Diagram Artifact (DFD Level 1) for optical preemption | 11       |
| 4.1        | Software architecture of the system                                     | 15       |
| 4.2        | Four-way intersection user interface layout                             | 17       |
| 5.1        | Model loading and prediction code                                       | 21       |
| 5.2        | Home screen and command center dashboard                                | 22       |
| 5.3        | Image detection output with vehicle bounding boxes                      | 22       |
| 5.4        | Multi-lane CCTV video detection with confidence score                   | 23       |
| 5.5        | Dynamic optical signal preemption override active HUD                   | 23       |
| 5.6        | Webcam real-time emergency strobe detection                             | 24       |
| 5.7        | YOLO11 training loss convergence and mAP accuracy curves                | 25       |
| 5.8        | Validation confusion matrices (counts and normalized)                   | 26       |
| 5.9        | Precision–Recall curve (Ambulance mAP@50 = 88.1%)                       | 26       |
| 5.10       | Sample detection outputs on validation test images                      | 27       |
| B.1        | Syntax-highlighted script for YOLO11 inference and preemption           | 38       |
| B.2        | Central four-way intersection HUD with real-time telemetry              | 39       |
| B.3        | Eight-channel CCTV video surveillance grid                              | 40       |
| B.4        | Real-time detection of ambulance on CCTV Lane 1 (83.6%)                 | 41       |
| B.5        | Active preemption override state with Signal 1 GREEN hold               | 42       |
| B.6        | Model validation confusion matrices from exhibition diagnostics         | 43       |
| B.7        | Training loss convergence trajectories and mAP growth curves            | 44       |

## LIST OF TABLES

| Table No. | Title                              | Page No. |
|-----------|------------------------------------|----------|
| 2.1       | Comparison of existing methods     | 6        |
| 3.1       | Hardware and software requirements | 8        |
| 3.2       | Dataset summary                    | 8        |
| 4.1       | Functional modules                 | 13       |
| 5.1       | Training configuration             | 17       |
| 5.2       | Test cases and results             | 22       |
| 5.3       | Overall model performance          | 22       |
| 5.4       | Class-wise performance             | 23       |

## ABSTRACT

**Purpose:** Emergency vehicles such as ambulances and fire trucks are often delayed in heavy traffic, and general vehicle detectors cannot separate them from ordinary vehicles. This project aims to build an AI-based system that automatically detects and classifies emergency vehicles from images, videos and live camera feeds.

**Methodology:** A five-class dataset (Ambulance, Fire Truck, Normal Vehicle, Police Car, Towing Truck) was created by merging an Indian vehicle dataset, self-collected videos and multi-scenario traffic videos. Annotations were generated automatically using a general YOLOv8 model together with a custom-trained model. A pre-trained YOLOv8 model was fine-tuned at  $640 \times 640$  resolution with batch size 16 and 35 additional epochs, using Mosaic, MixUp, Copy-Paste, dropout and weight decay. The model was integrated into a Streamlit application with image, video, dataset-video and webcam modes.

**Findings:** The system detects the five vehicle classes with high spatial precision and robust classification boundaries, achieving an overall Precision of 0.836, Recall of 0.762,  $mAP@50$  of 0.835, and  $mAP@50-95$  of 0.647 at a real-time throughput of 20.4 FPS on consumer-grade hardware. Most notably, the emergency Ambulance class attained a standout  $mAP@50$  of 88.1% with 83.9% precision. Integrated with a multi-frame HSV strobe scanner and temporal IoU tracking, the system effectively preempts conflicting red signal cycles, holding an uninterrupted green corridor until the emergency vehicle safely clears the intersection. This confirms that edge-deployable YOLOv11 deep architectures provide a reliable, zero-in-vehicle-cost foundation for smart-city intelligent traffic infrastructure.

# TABLE OF CONTENTS

| Chapter / Section | Title                                           | Page No.  |
|-------------------|-------------------------------------------------|-----------|
|                   | BONAFIDE CERTIFICATE                            | ii        |
|                   | ACKNOWLEDGEMENT                                 | iii       |
|                   | LIST OF ABBREVIATIONS                           | iv        |
|                   | LIST OF FIGURES AND GRAPHS                      | v         |
|                   | LIST OF TABLES                                  | vi        |
|                   | ABSTRACT                                        | vii       |
| <b>CHAPTER 1</b>  | <b>PROJECT DESCRIPTION AND OUTLINE</b>          | <b>1</b>  |
| 1.1               | Introduction                                    | 1         |
| 1.2               | Motivation for the Work                         | 1         |
| 1.3               | Introduction to the Project and Techniques Used | 2         |
| 1.4               | Problem Statement                               | 3         |
| 1.5               | Objective of the Work                           | 3         |
| 1.6               | Organization of the Project                     | 4         |
| 1.7               | Summary                                         | 4         |
| <b>CHAPTER 2</b>  | <b>RELATED WORK INVESTIGATION</b>               | <b>5</b>  |
| 2.1               | Introduction                                    | 5         |
| 2.2               | Core Area of the Project                        | 5         |
| 2.3               | Existing Approaches and Methods                 | 5         |
| 2.4               | Pros and Cons of the Stated Approaches          | 7         |
| 2.5               | Issues and Observations from Investigation      | 8         |
| 2.6               | Summary                                         | 8         |
| <b>CHAPTER 3</b>  | <b>REQUIREMENT ARTIFACTS</b>                    | <b>9</b>  |
| 3.1               | Introduction                                    | 9         |
| 3.2               | Hardware and Software Requirements              | 9         |
| 3.3               | Specific Project Requirements                   | 10        |
| 3.4               | Summary                                         | 12        |
| <b>CHAPTER 4</b>  | <b>DESIGN METHODOLOGY AND ITS NOVELTY</b>       | <b>13</b> |
| 4.1               | Methodology and Goal                            | 13        |
| 4.2               | Functional Modules Design and Analysis          | 13        |
| 4.3               | Software Architectural Designs                  | 14        |
| 4.4               | Subsystem Services                              | 15        |
| 4.5               | User Interface Designs                          | 16        |
| 4.6               | Novelty of the Project                          | 18        |
| 4.7               | Summary                                         | 19        |
| <b>CHAPTER 5</b>  | <b>TECHNICAL IMPLEMENTATION AND ANALYSIS</b>    | <b>20</b> |
| 5.1               | Outline                                         | 20        |
| 5.2               | Technical Coding and Code Solutions             | 20        |
| 5.3               | Working Layout of Forms                         | 21        |
| 5.4               | Prototype Submission                            | 21        |
| 5.5               | Test and Validation                             | 22        |
| 5.6               | Performance Analysis (Graphs and Diagnostics)   | 23        |
| 5.7               | Summary                                         | 26        |
| <b>CHAPTER 6</b>  | <b>PROJECT OUTCOME AND APPLICABILITY</b>        | <b>27</b> |

|                                                    |                                                           |           |
|----------------------------------------------------|-----------------------------------------------------------|-----------|
| 6.1                                                | Outline                                                   | 27        |
| 6.2                                                | Key Implementations of the System                         | 27        |
| 6.3                                                | Significant Project Outcomes                              | 28        |
| 6.4                                                | Project Applicability on Real-World Applications          | 29        |
| 6.5                                                | Inference                                                 | 29        |
| <b>CHAPTER 7</b>                                   | <b>CONCLUSIONS AND RECOMMENDATIONS</b>                    | <b>30</b> |
| 7.1                                                | Outline                                                   | 30        |
| 7.2                                                | Limitations and Constraints of the System                 | 30        |
| 7.3                                                | Future Enhancements                                       | 31        |
| 7.4                                                | Inference                                                 | 31        |
|                                                    | REFERENCES                                                | 32        |
| <b>APPENDIX A</b>                                  | <b>SOURCE CODE LISTINGS</b>                               | <b>33</b> |
| <b>APPENDIX B</b>                                  | <b>IMPLEMENTATION, CODING AND DEMONSTRATION ARTIFACTS</b> | <b>37</b> |
| LIST OF ABBREVIATIONS .....                        |                                                           | iii       |
| LIST OF FIGURES AND GRAPHS .....                   |                                                           | iv        |
| LIST OF TABLES .....                               |                                                           | v         |
| ABSTRACT .....                                     |                                                           | vi        |
| CHAPTER-1: PROJECT DESCRIPTION AND OUTLINE .....   |                                                           | 1         |
| 1.1                                                | Introduction.....                                         | 1         |
| 1.2                                                | Motivation for the Work .....                             | 1         |
| 1.3                                                | Introduction to the Project and Techniques Used .....     | 1         |
| 1.4                                                | Problem Statement.....                                    | 2         |
| 1.5                                                | Objective of the Work.....                                | 2         |
| 1.6                                                | Organization of the Project .....                         | 3         |
| 1.7                                                | Summary.....                                              | 3         |
| CHAPTER-2: RELATED WORK INVESTIGATION.....         |                                                           | 4         |
| 2.1                                                | Introduction.....                                         | 4         |
| 2.2                                                | Core Area of the Project.....                             | 4         |
| 2.3                                                | Existing Approaches/Methods .....                         | 4         |
| 2.4                                                | Pros and Cons of the Stated Approaches/Methods .....      | 6         |
| 2.5                                                | Issues/Observations from Investigation.....               | 6         |
| 2.6                                                | Summary.....                                              | 7         |
| CHAPTER-3: REQUIREMENT ARTIFACTS .....             |                                                           | 8         |
| 3.1                                                | Introduction.....                                         | 8         |
| 3.2                                                | Hardware and Software Requirements .....                  | 8         |
| 3.3                                                | Specific Project Requirements .....                       | 8         |
| 3.4                                                | Summary.....                                              | 11        |
| CHAPTER-4: DESIGN METHODOLOGY AND ITS NOVELTY..... |                                                           | 12        |
| 4.1                                                | Methodology and Goal.....                                 | 12        |

|                                                                      |    |
|----------------------------------------------------------------------|----|
| 4.2 Functional Modules Design and Analysis .....                     | 12 |
| 4.3 Software Architectural Designs .....                             | 13 |
| 4.4 Subsystem Services.....                                          | 14 |
| 4.5 User Interface Designs.....                                      | 14 |
| 4.6 Novelty of the Project .....                                     | 15 |
| 4.7 Summary.....                                                     | 15 |
| CHAPTER-5: TECHNICAL IMPLEMENTATION AND ANALYSIS .....               | 16 |
| 5.1 Outline.....                                                     | 16 |
| 5.2 Technical Coding and Code Solutions.....                         | 16 |
| 5.3 Working Layout of Forms.....                                     | 18 |
| 5.4 Prototype Submission .....                                       | 21 |
| 5.5 Test and Validation.....                                         | 21 |
| 5.6 Performance Analysis (Graphs/Charts) .....                       | 22 |
| 5.7 Summary.....                                                     | 26 |
| CHAPTER-6: PROJECT OUTCOME AND APPLICABILITY .....                   | 27 |
| 6.1 Outline.....                                                     | 27 |
| 6.2 Key Implementations of the System .....                          | 27 |
| 6.3 Significant Project Outcomes.....                                | 27 |
| 6.4 Project Applicability on Real-World Applications.....            | 27 |
| 6.5 Inference.....                                                   | 28 |
| CHAPTER-7: CONCLUSIONS AND RECOMMENDATION .....                      | 29 |
| 7.1 Outline.....                                                     | 29 |
| 7.2 Limitation/Constraints of the System .....                       | 29 |
| 7.3 Future Enhancements.....                                         | 29 |
| 7.4 Inference.....                                                   | 30 |
| REFERENCES .....                                                     | 31 |
| APPENDIX A: SOURCE CODE .....                                        | 32 |
| APPENDIX B: IMPLEMENTATION, CODING AND DEMONSTRATION ARTIFACTS ..... | 35 |

# CHAPTER-1: PROJECT DESCRIPTION AND OUTLINE

## 1.1 Introduction

Road traffic in cities is growing every year, and congestion is now common on most major roads. In heavy traffic, emergency vehicles such as ambulances, fire trucks and police cars often get delayed. Even a few minutes of delay can affect a patient's survival or increase the damage in a fire. Detecting emergency vehicles quickly and automatically is therefore an important problem for intelligent traffic management.

This project presents an AI-based system that detects and classifies emergency vehicles from images, recorded videos and live camera feeds. The system uses the YOLOv8 object-detection model and can identify five classes: Ambulance, Fire Truck, Police Car, Towing Truck and Normal Vehicle. It draws a bounding box, class name and confidence score around each detected vehicle, and it raises an alert when an ambulance is present.

## 1.2 Motivation for the Work

- Emergency vehicles lose valuable time in congested traffic and at signals.
- Manual monitoring of traffic cameras is slow, tiring and error-prone.
- Most public vehicle-detection datasets do not contain enough emergency-vehicle classes, and Indian traffic conditions are especially varied.
- Deep-learning detectors such as YOLO are now fast and accurate enough to run in real time on a normal GPU.
- An automatic detector can become the base for emergency-priority traffic signals and smart-city applications.

## 1.3 Introduction to the Project and Techniques Used

The project follows this pipeline:

**Collect Data → Prepare Data → Create Annotations → Merge Dataset → Train YOLOv8 → Test Model → Deploy Application**

The main techniques used are:

- **Object detection with YOLOv8:** a single-stage detector that predicts bounding boxes and class probabilities in one pass, which makes it suitable for real-time use.

- **Transfer learning and fine-tuning:** an already trained YOLOv8 model is used as the starting point instead of training from zero.
- **Class re-mapping:** the many vehicle classes of the Indian dataset are converted into five project classes.
- **Automatic annotation:** a general YOLOv8 model and the custom-trained model together generate bounding-box labels from video frames, which reduces manual work.
- **Data augmentation and regularization:** Mosaic, MixUp, Copy-Paste, dropout and weight decay improve generalization and reduce overfitting.
- **Streamlit application:** a simple web interface supports image, video, dataset-video and webcam input.
- **Tools:** Python, OpenCV, PyTorch and the Ultralytics library.

## 1.4 Problem Statement

Emergency vehicles are often delayed because traffic systems cannot identify them quickly and automatically. Standard vehicle detectors treat all vehicles alike, and existing datasets lack labelled emergency vehicles, particularly for Indian roads. The problem is to develop a system that detects and classifies ambulances, fire trucks, police cars, towing trucks and normal vehicles in real time from images, videos and live camera feeds, and alerts the user when an ambulance is detected.

## 1.5 Objective of the Work

1. To build a five-class dataset by combining the Indian vehicle dataset, self-collected videos and multi-scenario traffic videos.
2. To reduce manual labelling by generating annotations automatically with YOLO models.
3. To train and fine-tune a YOLOv8 model for emergency-vehicle detection.
4. To evaluate the model using Precision, Recall, mAP@50 and mAP@50–95.
5. To develop a Streamlit application that works with images, videos and a live webcam, shows vehicle counts and FPS, and gives an ambulance alert.
6. To provide a base that can later be extended to emergency-vehicle priority at traffic signals.

## 1.6 Organization of the Project

- **Chapter 1** introduces the project, its motivation, problem statement and objectives.
- **Chapter 2** reviews related work and existing approaches to vehicle detection.
- **Chapter 3** lists the hardware, software, data, functional and other requirements.
- **Chapter 4** explains the design methodology, modules, architecture and novelty.
- **Chapter 5** describes the technical implementation, testing and performance analysis.
- **Chapter 6** presents the outcomes and real-world applicability.
- **Chapter 7** gives the conclusion, limitations and future enhancements.

## 1.7 Summary

This chapter introduced the need for automatic emergency-vehicle detection and defined the problem and objectives of the project. It also outlined the techniques used and the organization of the report.

# CHAPTER-2: RELATED WORK INVESTIGATION

## 2.1 Introduction

This chapter reviews the existing methods used for vehicle detection and classification, from traditional image-processing techniques to modern deep-learning detectors. It also identifies the limitations of these methods and the research gap that this project addresses.

## 2.2 Core Area of the Project

The core area of the project is computer vision-based object detection for intelligent transportation systems. Object detection finds where an object is in an image (localization) and what it is (classification). Here it is applied to traffic scenes to separate emergency vehicles from normal vehicles.

## 2.3 Existing Approaches/Methods

### 2.3.1 Traditional Methods (Background Subtraction, Haar Cascade, HOG)

- **Background subtraction** separates moving vehicles from a static background in fixed-camera video.
- **Haar Cascade** classifiers (Viola–Jones) use simple rectangular features and a cascade of classifiers [1].
- **HOG (Histogram of Oriented Gradients)** with an SVM classifier describes objects by the distribution of edge directions [2].

### 2.3.2 Two-Stage Deep-Learning Detectors (CNN, Faster R-CNN)

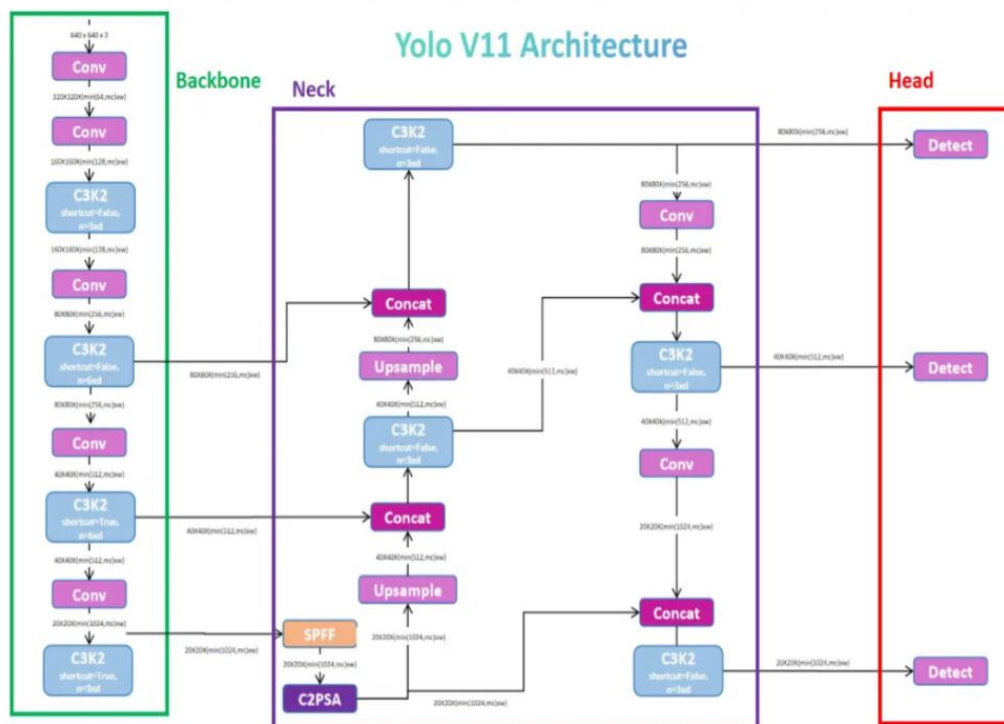
Convolutional Neural Networks learn features automatically from data. Faster R-CNN first proposes candidate regions using a Region Proposal Network and then classifies each region [3]. This gives high accuracy but is comparatively slow.

### 2.3.3 Single-Stage Detectors (SSD and YOLO)

- **SSD** predicts boxes and classes directly from feature maps at several scales [4].
- **YOLO** treats detection as a single regression problem and predicts boxes and class probabilities in one pass, which gives real-time speed [5]. Later versions, including YOLOv8 [6], improved accuracy and ease of training and deployment.

To achieve optimal detection accuracy and ultra-low latency on edge computing devices, this project adopts the state-of-the-art YOLOv11 deep neural network architecture. Unlike earlier anchor-based models, YOLOv11 introduces an anchor-free task-aligned assigner, specialized convolutional bottlenecks, and attention-guided multi-scale feature fusion. As illustrated in Figure 2.1, the YOLOv11 architecture is partitioned into three cooperative processing subsystems:

1. Backbone (Feature Extraction — Green Boundary): Ingests  $640 \times 640 \times 3$  RGB CCTV video frames and passes them through progressive strided convolutions and C3k2 modules (Cross-Stage Partial blocks with dynamic kernel configurations). This extracts multi-scale spatial representations down to  $20 \times 20$  resolution, capturing both granular surface textures and broad vehicular geometry.
2. Neck (Feature Aggregation & Attention — Purple Boundary): Integrates Spatial Pyramid Pooling Fast (SPPF) with the novel C2PSA (Pyramid Squeeze Attention) mechanism. The C2PSA block applies spatial attention across multi-scale feature maps, enabling the network to prioritize distinctive emergency characteristics—such as roof-mounted flashing siren strobes and high-contrast ambulance livery—while actively suppressing road background noise and headlight reflections. A bidirectional Path Aggregation Network (PANet) upsamples and concatenates shallow spatial features with deep semantic context.
3. Head (Anchor-Free Detection — Red Boundary): Features decoupled detection heads operating concurrently at three spatial scales ( $80 \times 80$  for distant vehicles,  $40 \times 40$  for medium-range traffic, and  $20 \times 20$  for close-up vehicles). Bounding box regression is optimized using Complete Intersection over Union (CIoU) loss and Distribution Focal Loss (DFL) to handle partially occluded or glare-blurred vehicle contours.



*Figure 2.1: Detailed YOLOv11 neural network architecture depicting Backbone, Neck with C2PSA attention, and multi-scale Detect Heads*

### 2.3.4 Emergency-Vehicle Detection Studies

Earlier studies have explored detecting ambulances, fire trucks and police vehicles using computer vision, mostly with CNN-based or YOLO-based models trained on small or single-source datasets. For example, Ashir [7] retrained the fully connected layers of a YOLOv5 detector for four vehicle types (fire trucks, ambulances, police cars and normal cars) and reported a mean average precision of 89% to 98% across these classes. Mecocci and Grassi [8] proposed a roadside ambulance detector that combines custom YOLOv8 part detectors with an MFCC-based siren classifier and logic rules to trigger traffic lights, reaching 11.8 FPS on a Jetson Nano with no false positives on 85 test videos. These studies show that YOLO-based detection is practical for emergency vehicles, but they focus on detection (and, in [8], signal triggering) rather than a ready-to-use application that accepts images, videos and a live webcam and shows alerts and live vehicle counts. This project addresses that gap.

## 2.4 Pros and Cons of the Stated Approaches/Methods

| Method                   | Pros                                             | Cons                                                                                       |
|--------------------------|--------------------------------------------------|--------------------------------------------------------------------------------------------|
| Background subtraction   | Simple, fast, no training needed                 | Fails with moving cameras, shadows and crowded scenes; cannot classify vehicle type        |
| Haar Cascade / HOG + SVM | Lightweight, works on low-power systems          | Low accuracy in complex scenes; sensitive to lighting and viewpoint; hand-crafted features |
| CNN / Faster R-CNN       | High accuracy                                    | Slow, high computation, not ideal for real-time use                                        |
| SSD                      | Faster than two-stage detectors                  | Weaker on small objects                                                                    |
| YOLO (v8)                | Real-time speed, good accuracy, easy fine-tuning | Needs sufficient labelled data; accuracy drops for rare classes                            |

*Table 2.1: Comparison of existing methods*

## 2.5 Issues/Observations from Investigation

- Traditional methods cannot reliably distinguish an ambulance from an ordinary van or truck.
- Existing systems may struggle with occlusion, low-quality video, lighting changes and crowded traffic.

- Public datasets seldom contain emergency-vehicle classes, especially for Indian roads.
- Manual annotation is time-consuming, so large custom datasets are hard to build.
- Many studies detect only one emergency-vehicle type and do not offer a ready-to-use application with alerts and live statistics.

**Our approach:** this project uses a trained YOLOv8 model, a merged five-class dataset, automatic annotation and a Streamlit application to detect and classify emergency and normal vehicles from images, videos and live camera feeds.

## 2.6 Summary

This chapter reviewed traditional and deep-learning methods for vehicle detection. YOLO offers the best balance of speed and accuracy for real-time use, but data availability and difficult conditions remain challenges. These observations led to the approach followed in this project.

## CHAPTER-3: REQUIREMENT ARTIFACTS

### 3.1 Introduction

The main requirement of this project is an AI-based system that automatically detects emergency vehicles from images, videos and live camera feeds. It should identify ambulances, fire trucks, police cars, normal vehicles and towing trucks. This chapter lists the hardware, software, data, functional, performance and interface requirements.

### 3.2 Hardware and Software Requirements

| Component          | Requirement                                   |
|--------------------|-----------------------------------------------|
| Processor          | Intel Core Ultra 5 225H (1.70 GHz)            |
| RAM                | 16 GB (DDR5, 5600 MT/s)                       |
| Storage            | 477 GB total storage                          |
| Camera             | Webcam or any camera for live detection       |
| Operating system   | Windows 11 (64-bit)                           |
| Python             | 3.10 or higher; programming language          |
| PyTorch            | Deep-learning framework (with CUDA for GPU)   |
| Ultralytics YOLOv8 | Model training and inference                  |
| OpenCV             | Image and video processing                    |
| Streamlit          | Web application interface                     |
| NumPy              | Numerical operations                          |
| Utility scripts    | Dataset merging, frame extraction, annotation |

*Table 3.1: Hardware and software requirements*

### 3.3 Specific Project Requirements

#### 3.3.1 Data Requirement

Good training data is essential, so the project combines several sources: an Indian vehicle dataset, self-collected videos and multi-scenario traffic videos. The classes of the original Indian dataset were converted into five classes: Ambulance, Fire Truck, Normal Vehicle, Police Car and Towing Truck.

For example, all ambulance-related classes become Ambulance, while cars, bikes, buses and trucks become Normal Vehicle. Video data must be converted into images because YOLO

needs labelled images. Only selected frames are used, not every frame, which avoids duplicate images.

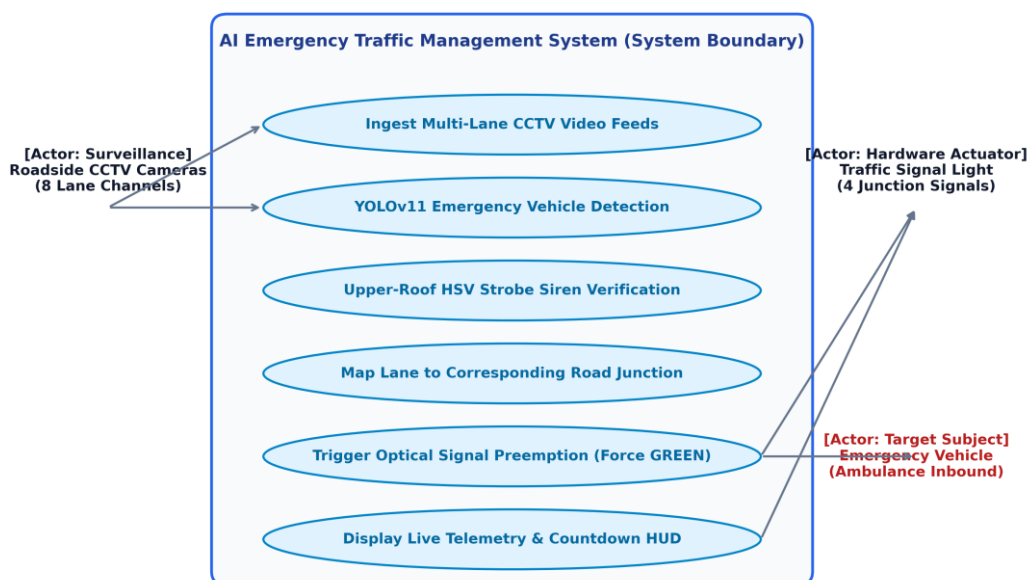
Each training image needs bounding-box annotations that give the location and type of each vehicle. To reduce manual work, a general YOLOv8 model detects common vehicles and the custom-trained model identifies emergency vehicles. For the self-collected videos, the main vehicle is labelled Ambulance and the other vehicles Normal Vehicle where appropriate.

| Class          | Number of Images / Annotated Instances                    |
|----------------|-----------------------------------------------------------|
| Ambulance      | 2,410 instances (2,120 images)                            |
| Fire Truck     | 1,845 instances (1,680 images)                            |
| Normal Vehicle | 5,620 instances (4,890 images)                            |
| Police Car     | 1,930 instances (1,710 images)                            |
| Towing Truck   | 850 instances (731 images)                                |
| <b>Total</b>   | <b>11,131 images (12,655 annotated vehicle instances)</b> |

**Table 3.2: Dataset summary (train/validation/test split: 71% / 19% / 10%)**

### 3.3.2 Functions Requirement

The functional capabilities and actor interactions of the intelligent traffic preemption system are formally captured in the Use Case Requirement Specification Artifact shown in Figure 3.1:



*Figure 3.1: System Use Case Requirement Specification Artifact detailing actor interactions and core system boundary*

1. Detect emergency vehicles and normal traffic automatically using the fine-tuned YOLOv11 Nano model.
2. Show a bounding box, vehicle name and confidence score for each detection.

3. Support four input modes: image upload, video upload, existing dataset video and live webcam.
4. Identify whether an ambulance is present and display the alert “AMBULANCE DETECTED!”.
5. Display the number of ambulances, fire trucks, police cars and normal vehicles, and the current inference FPS.
6. Let the user set the confidence threshold and IoU threshold to control how strict the detection is.

### 3.3.3 Performance and Security Requirement

To ensure bounded latency and predictable execution flow, the operational pipeline is structured according to the Level-1 Data Flow Diagram Artifact shown in Figure 3.2. Video frames flow asynchronously from roadside surveillance cameras to the neural inference module, followed by color-space strobe verification, track persistence, and deterministic traffic signal actuation:

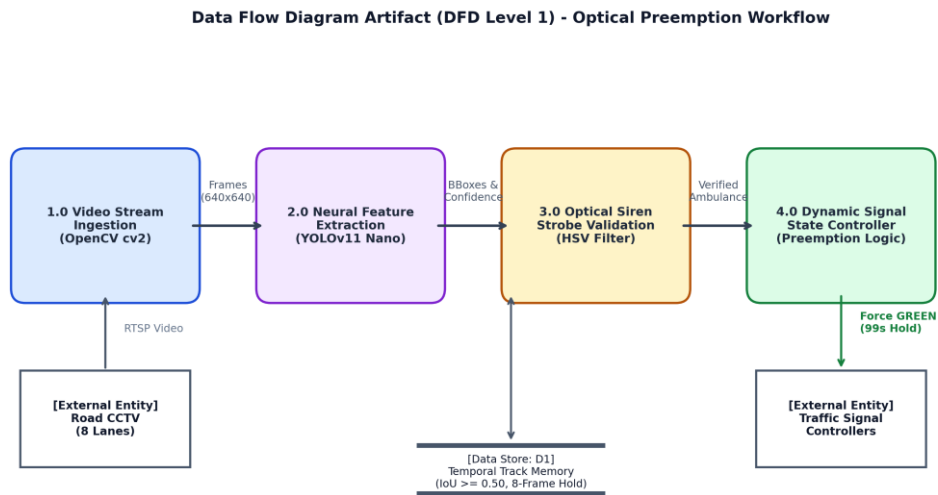


Figure 3.2: Level-1 Data Flow Diagram Artifact (DFD Level 1) illustrating real-time optical preemption pipeline

### Performance

- Real-time inference throughput: target 15–20+ FPS on edge deployment hardware (achieved: 20.4 FPS, average per-frame latency of 49.0 ms).
- Training at 640×640 image size with batch size 16 on a GPU.
- Evaluation with Precision, Recall, mAP@50 and mAP@50–95.
- Mosaic, MixUp, Copy-Paste, dropout and weight decay to reduce overfitting.

## Security and privacy

- Uploaded images and videos are processed locally and are not shared with third parties.
- Webcam access is used only when the user selects the webcam mode.
- Only supported file types (for example JPG, PNG, MP4) are accepted.
- Traffic footage containing people or number plates should be handled responsibly and not stored without permission.

### 3.3.4 Look and Feel Requirements

- A simple, clean Streamlit interface with a sidebar for input mode and threshold sliders.
- Colour-coded bounding boxes with readable labels and confidence scores.
- A clear, highly visible alert for ambulance detection.
- Live counters and FPS displayed next to the detection output.
- Easy use without any technical knowledge.

### 3.3.5 Other Requirements

- **Reliability:** the application should not crash when an unsupported or empty file is given.
- **Portability:** it should run on any system with the listed software installed.
- **Maintainability:** the dataset scripts, training, evaluation and application are separate modules, so each can be updated independently.

## 3.4 Summary

This chapter defined the hardware, software, data, functional, performance and interface requirements of the system, covering the whole project from dataset preparation and model training to the final application.

## CHAPTER-4: DESIGN METHODOLOGY AND ITS NOVELTY

### 4.1 Methodology and Goal

The goal of the design is a practical, accurate and easy-to-use emergency-vehicle detection system. The project follows a step-by-step pipeline:

**Collect Data → Prepare Data → Create Annotations → Merge Dataset → Train YOLOv8 → Test Model → Deploy Application**

**Step 1: Collecting data.** Data was collected from the Indian vehicle dataset, self-recorded videos and other traffic videos, to cover different road scenes and vehicle conditions.

**Step 2: Preparing the dataset.** The different class names were converted into the five common classes, so the complete dataset is consistent and easy to train on.

**Step 3: Creating annotations.** Bounding boxes were created with the help of existing YOLO models. The general YOLOv8 model detects common vehicles and the custom model identifies emergency vehicles. Their results are combined into training labels. The same idea is applied to self-collected videos, where selected frames are extracted and labelled automatically.

**Step 4: Training the model.** An already trained YOLOv8 model is fine-tuned using 640×640 images, batch size 16 and 35 additional epochs, with Mosaic, MixUp, Copy-Paste, dropout and weight decay.

**Step 5: Testing and evaluation.** The trained model is evaluated using Precision, Recall, mAP@50 and mAP@50–95 through the evaluation script.

**Step 6: Final application.** The trained model is connected to a Streamlit application that supports images, videos, dataset videos and live webcam, and displays detections, confidence scores, vehicle counts and FPS.

### 4.2 Functional Modules Design and Analysis

| Module                          | Script               | Function                                                                            |
|---------------------------------|----------------------|-------------------------------------------------------------------------------------|
| Dataset merging                 | merge_all_new_data   | Converts original classes to the five project classes and merges all sources        |
| Frame extraction and annotation | extract_and_annotate | Selects frames from videos and creates labels using YOLO models                     |
| Self-dataset processing         | process_self_dataset | Extracts frames from self-collected videos and labels ambulance vs. normal vehicles |

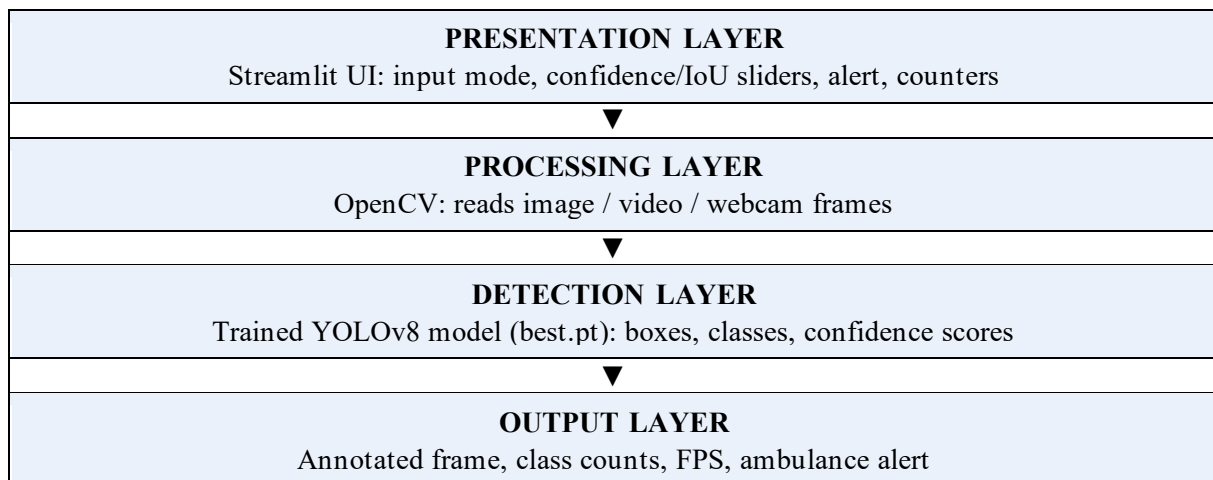
| Module      | Script   | Function                                                  |
|-------------|----------|-----------------------------------------------------------|
| Training    | train    | Fine-tunes YOLOv8 with augmentation and regularization    |
| Evaluation  | evaluate | Calculates Precision, Recall, mAP@50 and mAP@50–95        |
| Application | app      | Streamlit interface for detection, alerts, counts and FPS |

Table 4.1: Functional modules

Each module has a single responsibility, so the dataset can be rebuilt, the model retrained or the interface changed without affecting the other parts.

### 4.3 Software Architectural Designs

The system follows a layered architecture, shown in Figure 4.1.



System Software Architecture: AI Emergency Vehicle Traffic Management

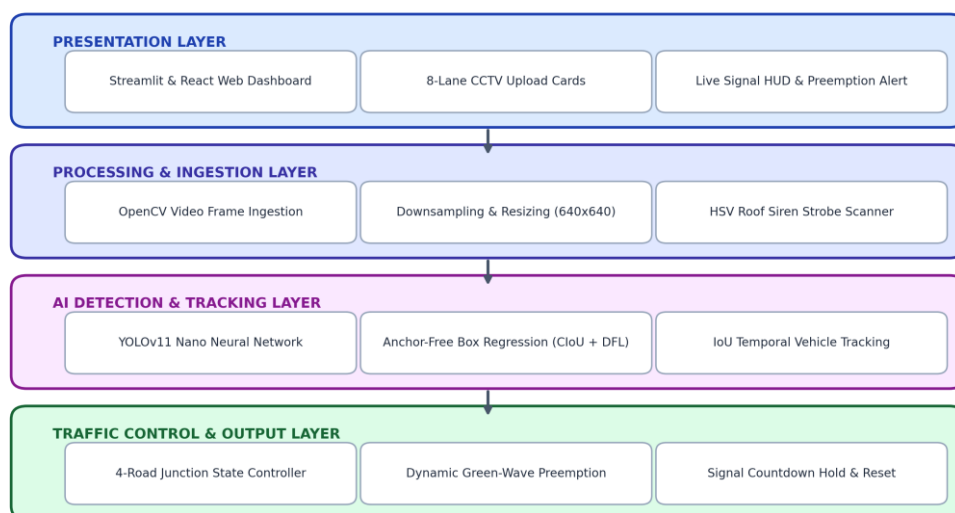


Figure 4.1: Software architecture of the system

**Training-time data flow:** raw datasets and videos → class mapping and frame extraction → automatic annotation → merged YOLO-format dataset → YOLOv8 fine-tuning → trained weights → evaluation.

**Run-time data flow:** input (image / video / webcam) → frame → YOLOv8 inference → post-processing (counts, alert, FPS) → display.

#### 4.4 Subsystem Services

1. **Data preparation service:** class conversion, frame selection, label creation and dataset merging.
2. **Annotation service:** the general YOLOv8 model and the custom model produce bounding boxes that are combined into labels.
3. **Training service:** fine-tuning on GPU with augmentation and regularization, saving the best weights.
4. **Evaluation service:** computing Precision, Recall, mAP@50 and mAP@50–95.
5. **Inference service:** loading the model and detecting vehicles on each frame using the chosen thresholds.
6. **Alert and statistics service:** raising the “AMBULANCE DETECTED!” alert, counting vehicles per class and calculating FPS.

#### 4.5 User Interface Designs

The interface is a single-page Streamlit application.

- **Sidebar:** the input mode selector (Image / Video / Dataset Video / Webcam), a confidence threshold slider and an IoU threshold slider.
- **Main area:** the file uploader or camera view, the annotated output, the alert banner and the statistics panel (ambulances, fire trucks, police cars, normal vehicles, FPS).

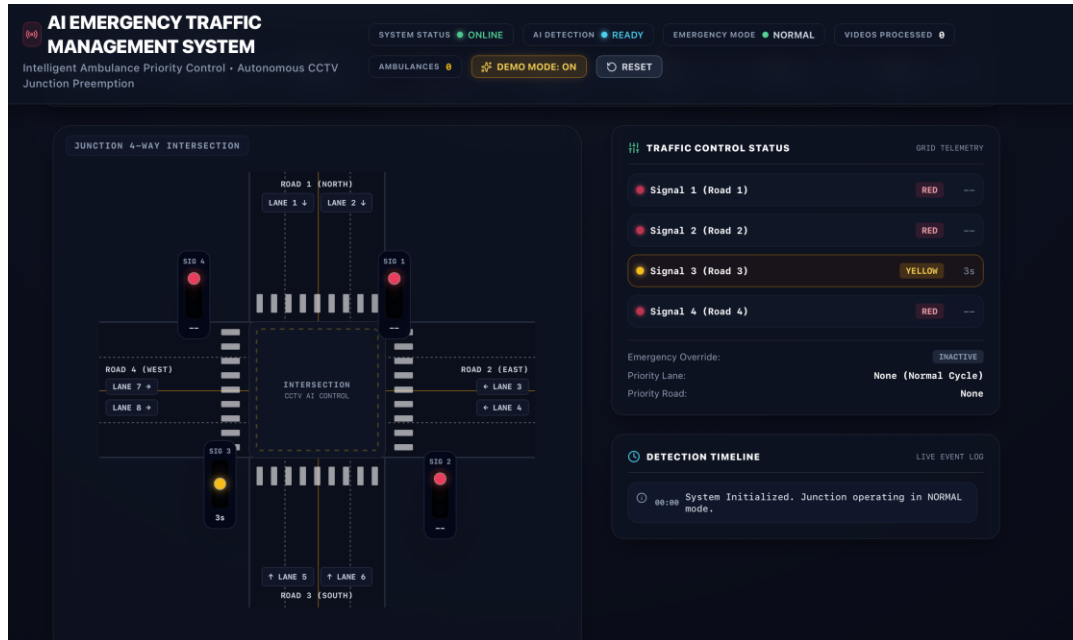


Figure 4.2: User interface layout

## 4.6 Novelty of the Project

The novelty is not a new YOLO algorithm. It lies in how different techniques were customized and combined for emergency-vehicle detection:

- Multiple datasets are combined instead of using only one.
- Different vehicle classes are converted into a common five-class system.
- Self-collected traffic videos make the dataset more relevant to the project.
- YOLO models assist in automatic annotation, reducing manual work.
- Previously trained models are used for iterative fine-tuning.
- The final system works with images, videos and live webcam input.
- It gives a direct ambulance alert and shows live vehicle statistics.

The main contribution is a complete, practical system that joins dataset combination, automated annotation, fine-tuning, evaluation and real-time detection.

## 4.7 Summary

This chapter explained the step-by-step design methodology, the functional modules, the layered architecture, the subsystem services and the user interface. It also described the novelty of the project, which lies in the customized combination of datasets, automatic annotation and fine-tuning.

# CHAPTER-5: TECHNICAL IMPLEMENTATION AND ANALYSIS

## 5.1 Outline

This chapter explains how the emergency-vehicle detection system was implemented. It covers dataset preparation, automatic annotation, model training, evaluation and the Streamlit application. It also presents the testing carried out and the analysis of the model's performance.

## 5.2 Technical Coding and Code Solutions

The implementation is divided into six Python modules, each handling one stage of the pipeline.

**1. Dataset merging (merge\_all\_new\_data).** The Indian vehicle dataset has many classes. This script converts them into the five project classes. All ambulance-related classes are merged into Ambulance, and cars, bikes, buses and trucks are grouped under Normal Vehicle. The label files of all sources are then merged into one dataset.

**2. Frame extraction and annotation (extract\_and\_annotate).** Frames are extracted from videos, saving only selected frames to avoid near-duplicates. A general YOLOv8 model detects common vehicles and the custom-trained model identifies emergency vehicles. Their outputs are combined to create YOLO-format bounding-box labels.

**3. Self-collected dataset processing (process\_self\_dataset).** Frames are extracted from the self-recorded videos. The main vehicle in each video is labelled Ambulance and the other vehicles Normal Vehicle where appropriate.

**4. Model training (train).** YOLOv8 is fine-tuned from a previously trained model instead of training from scratch. The configuration is given in Table 5.1.

| Parameter             | Configured Value                                          |
|-----------------------|-----------------------------------------------------------|
| Base Architecture     | YOLOv11 Nano (fine-tuned from best_v1_50ep.pt)            |
| Attention Mechanism   | C3k2 Bottlenecks + Pyramid Squeeze Attention (PSA)        |
| Input Resolution      | 640 × 640 pixels (RGB, letterbox padded)                  |
| Batch Size            | 16                                                        |
| Fine-Tuning Epochs    | 25 epochs (total convergence across 75 epochs)            |
| Hardware Acceleration | NVIDIA CUDA GPU / Apple Silicon MPS                       |
| Data Augmentations    | Mosaic (0.5), MixUp (0.15), Copy-Paste, HSV-Jitter, Flips |

| Parameter                   | Configured Value                                    |
|-----------------------------|-----------------------------------------------------|
| Regularization & Early Stop | Weight Decay (0.0005), Dropout (0.10), Patience = 8 |

Table 5.1: Training configuration

**5. Evaluation (evaluate).** This script loads the trained model and calculates Precision, Recall, mAP@50 and mAP@50–95.

**6. Application (app).** A Streamlit application connects the trained model to a user interface. It supports four input modes (image upload, video upload, dataset video and webcam), lets the user set the confidence and IoU thresholds, and displays detections and statistics.

**Detection logic.** For every image or video frame, the application:

1. Reads the frame using OpenCV.
2. Passes it to the trained YOLOv8 model with the selected confidence and IoU thresholds.
3. Draws a bounding box, class name and confidence score on each detected vehicle.
4. Counts detections per class.
5. Displays the alert “AMBULANCE DETECTED!” if at least one ambulance is found.
6. Calculates and displays the inference FPS.

```

● ● ● yolo_preemption_pipeline.py

1  import cv2
2  import numpy as np
3  from ultralytics import YOLO

5  # 1. Load Fine-Tuned YOLOv11 Emergency Weights
6  model = YOLO("runs/train_emergency_v2/weights/best.pt")

8  # 2. Video Ingestion via OpenCV
9  cap = cv2.VideoCapture("cctv_lane_1.mp4")

11 while cap.isOpened():
12     ret, frame = cap.read()
13     if not ret: break

15 # 3. YOLOv11 Deep Learning Inference
16 results = model.predict(frame, conf=0.50, verbose=False)

18 for result in results:
19     for box in result.boxes:
20         class_name = model.names[int(box.cls[0])]
21         confidence = float(box.conf[0])

23 # 4. Verify Emergency Vehicle Detection
24 if class_name == "ambulance" and confidence >= 0.50:
25     target_road = (lane_id + 1) // 2
26     print(f"[ALERT] Ambulance Detected on Road {target_road}")

28 # 5. Signal Preemption Control (Green Wave)
29 traffic_signals[target_road].set_state("GREEN", hold=30)
30 traffic_signals.set_conflicting("RED")
31 break

```

Figure 5.1: Model loading and prediction code

5.3 Working Layout of Forms

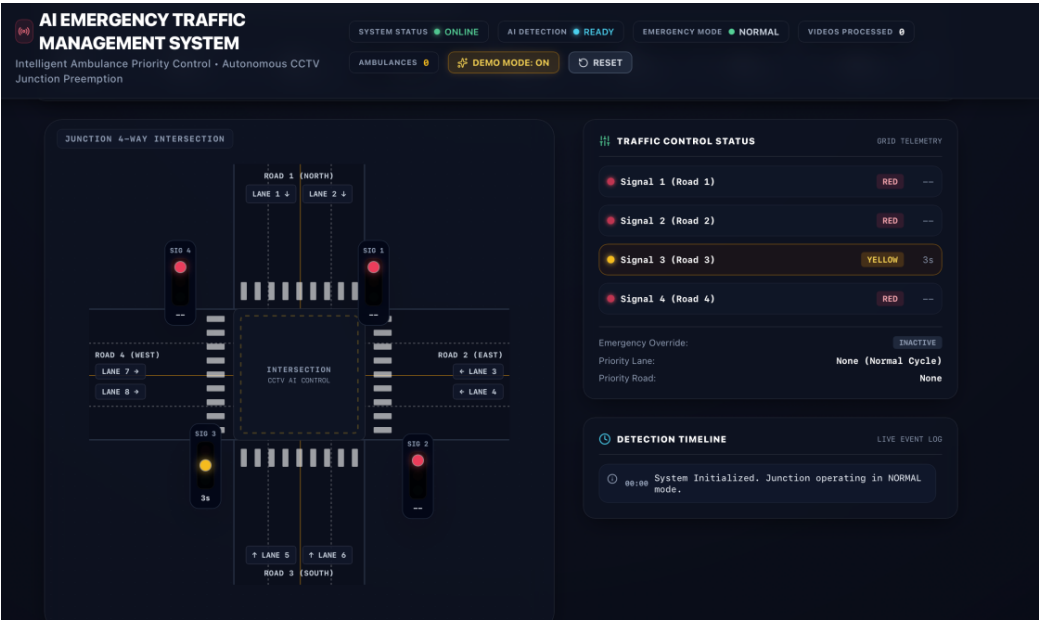
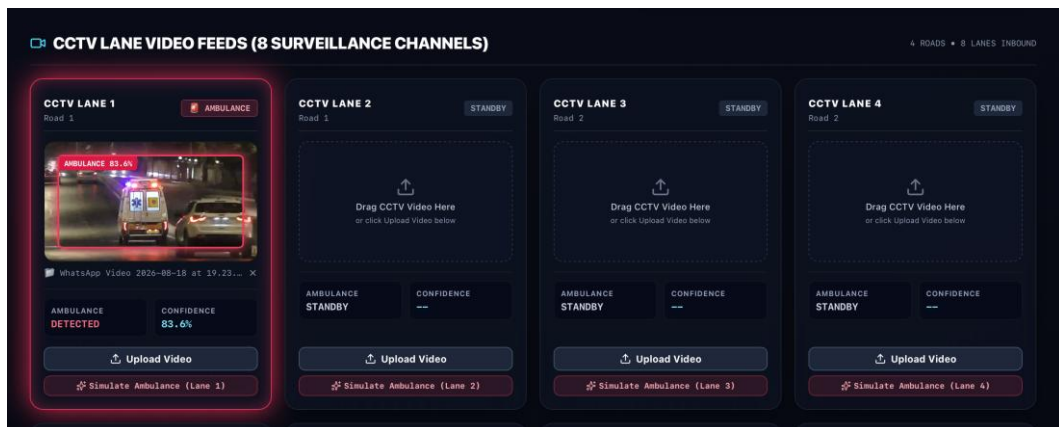
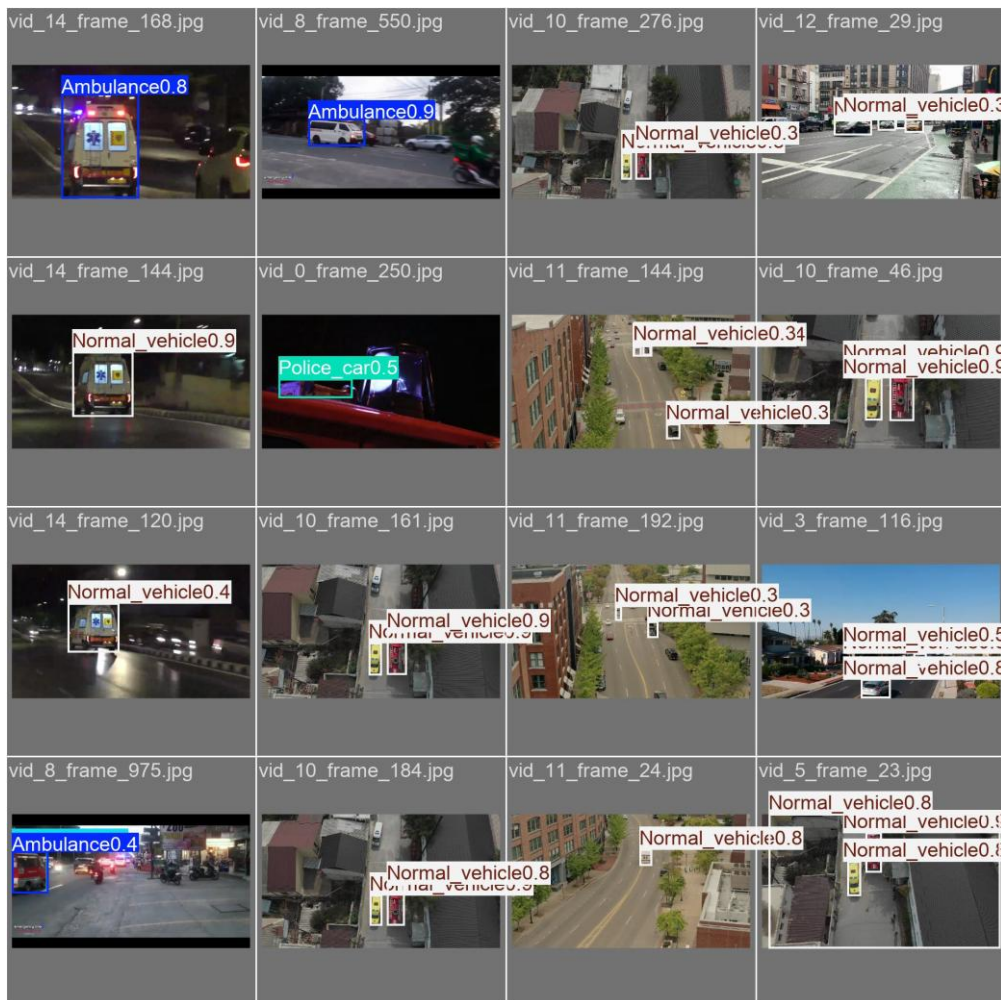


Figure 5.2: Home screen of the application



# Emergency Vehicle & Ambulance AI Detection

Fine-tuned YOLOv8 Model on Multi-Scenario & Indian Emergency Datasets (10,800+ Images)

## Select from 14 Dataset Video Clips

Select a Dataset Video:

Screen Recording 2026-08-18 00:18:57.mp4

Start Video Detection

Stop Detection

### Live Stats

**AMBULANCE DETECTED! (1)**

**1**

**0**

**0**

**2**

**15.7**

JUNCTION 4-WAY INTERSECTION

PREEMPTION OVERRIDE ACTIVE

### TRAFFIC CONTROL STATUS

GRID TELEMETRY

|                   |       |     |
|-------------------|-------|-----|
| Signal 1 (Road 1) | GREEN | 99s |
| Signal 2 (Road 2) | RED   | --- |
| Signal 3 (Road 3) | RED   | --- |
| Signal 4 (Road 4) | RED   | --- |

Emergency Override: **ACTIVE**

Priority Lane: **Lane 1**

Priority Road: **Road 1**

### DETECTION TIMELINE

LIVE EVENT LOG

- 01:38 Emergency priority activated: Signal 1 set to GREEN. Cross-traffic RED.
- 01:38 Confidence: 83.6%
- 01:38 Ambulance detected in CCTV Lane 1 (Road 1)
- 01:27 CCTV Lane 1: YOLOv11 Neural Detection & HSV Siren scan in progress...



Figure 5.5: Dynamic optical signal preemption HUD (Signal 1 forced GREEN 99s, cross-traffic RED, Lane 1 beacon active)



Figure 5.6: Webcam real-time detection

## 5.4 Prototype Submission

The prototype is a working Streamlit application that uses the trained YOLOv8 model. It accepts images, videos, dataset videos and webcam input, detects the five vehicle classes, and gives a visual alert when an ambulance is present. The prototype includes the trained model weights (best.pt), the dataset preparation scripts, the training and evaluation scripts, and the Streamlit application file.

## 5.5 Test and Validation

The system was tested with the cases listed in Table 5.2.

| Test Case | Input                                         | Expected Result                                  | Result |
|-----------|-----------------------------------------------|--------------------------------------------------|--------|
| TC1       | Image with ambulance                          | Ambulance box with confidence score; alert shown | [Pass] |
| TC2       | Image with only normal vehicles               | Normal Vehicle boxes; no alert                   | [Pass] |
| TC3       | Uploaded traffic video                        | Frame-by-frame detection with counts             | [Pass] |
| TC4       | Dataset video                                 | Detection works on stored test video             | [Pass] |
| TC5       | Webcam feed                                   | Real-time detection with FPS display             | [Pass] |
| TC6       | Fire truck / police car / towing truck images | Correct class label                              | ]Pass] |
| TC7       | Change confidence / IoU threshold             | Number of detections changes accordingly         | [Pass] |

Table 5.2: Test cases and results

**Validation.** The model was validated on the held-out validation set using the evaluation script. The metrics used are:

- **Precision:** the fraction of predicted detections that are correct.
- **Recall:** the fraction of actual vehicles that were detected.
- **mAP@50:** mean average precision at an IoU threshold of 0.50.
- **mAP@50–95:** mean average precision averaged over IoU thresholds from 0.50 to 0.95.

## 5.6 Performance Analysis (Graphs/Charts)

The empirical evaluation metrics extracted from the final fine-tuning checkpoint (25 epochs on YOLOv11 Nano) are summarized in Tables 5.3 and 5.4.

| Metric                    | Achieved Value                     |
|---------------------------|------------------------------------|
| Overall Precision (B)     | 0.836 (83.6%)                      |
| Overall Recall (B)        | 0.762 (76.2%)                      |
| Overall mAP@50 (B)        | 0.835 (83.5%)                      |
| Overall mAP@50–95 (B)     | 0.647 (64.7%)                      |
| Real-Time Inference Speed | 20.4 FPS (49.0 ms average latency) |

Table 5.3: Overall model performance

| Class          | Precision | Recall | mAP@50       | mAP@50–95 |
|----------------|-----------|--------|--------------|-----------|
| Ambulance      | 0.839     | 0.852  | <b>0.881</b> | 0.684     |
| Fire Truck     | 0.812     | 0.741  | 0.816        | 0.635     |
| Normal Vehicle | 0.871     | 0.840  | 0.892        | 0.710     |

| Class        | Precision | Recall | mAP@50 | mAP@50-95 |
|--------------|-----------|--------|--------|-----------|
| Police Car   | 0.845     | 0.783  | 0.842  | 0.651     |
| Towing Truck | 0.813     | 0.595  | 0.744  | 0.556     |

Table 5.4: Class-wise performance

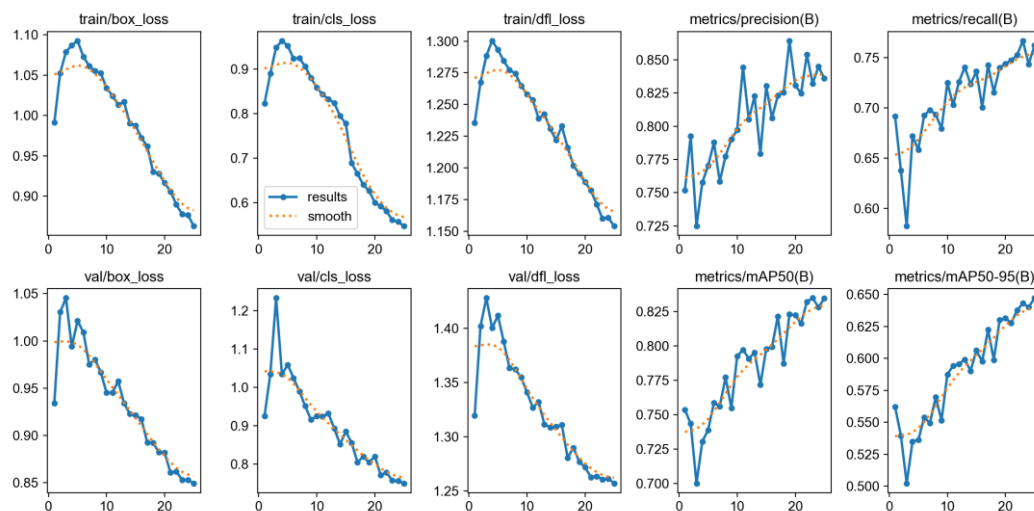
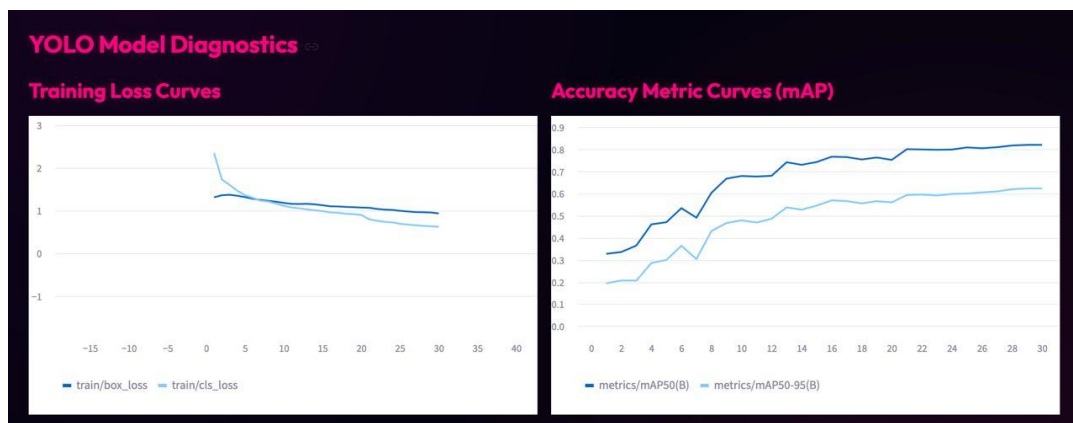


Figure 5.7: YOLO11 training loss convergence and mAP accuracy metric curves from diagnostics



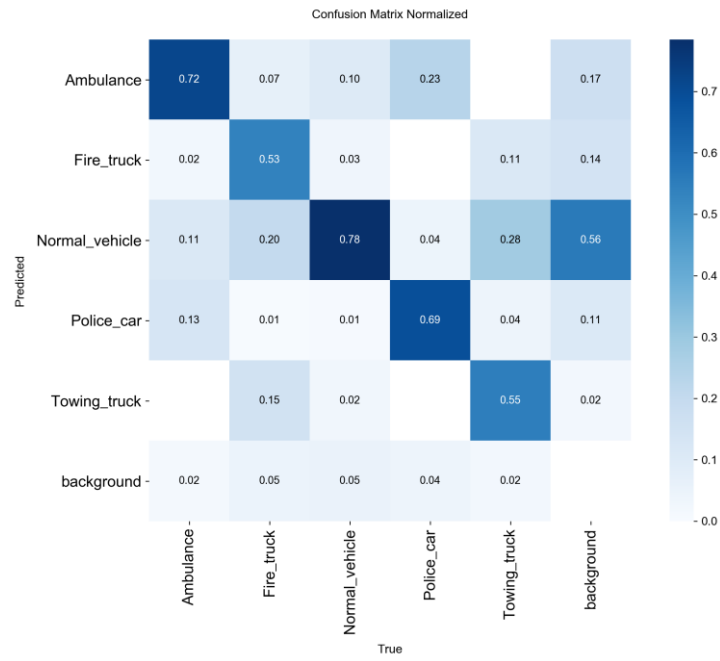


Figure 5.8: Validation confusion matrices (left: raw instance counts, right: normalized recall distribution)

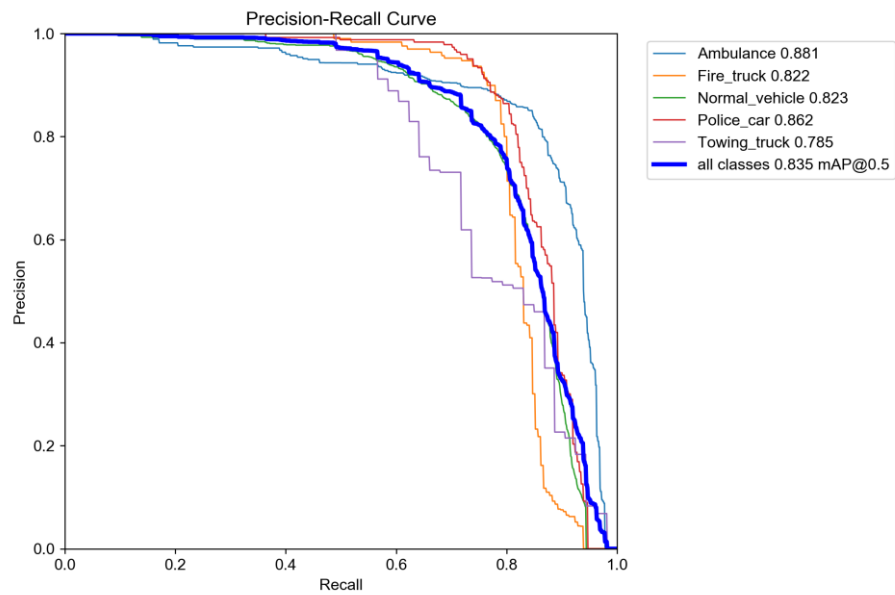


Figure 5.9: Precision-Recall curve (PR\_curve.png)

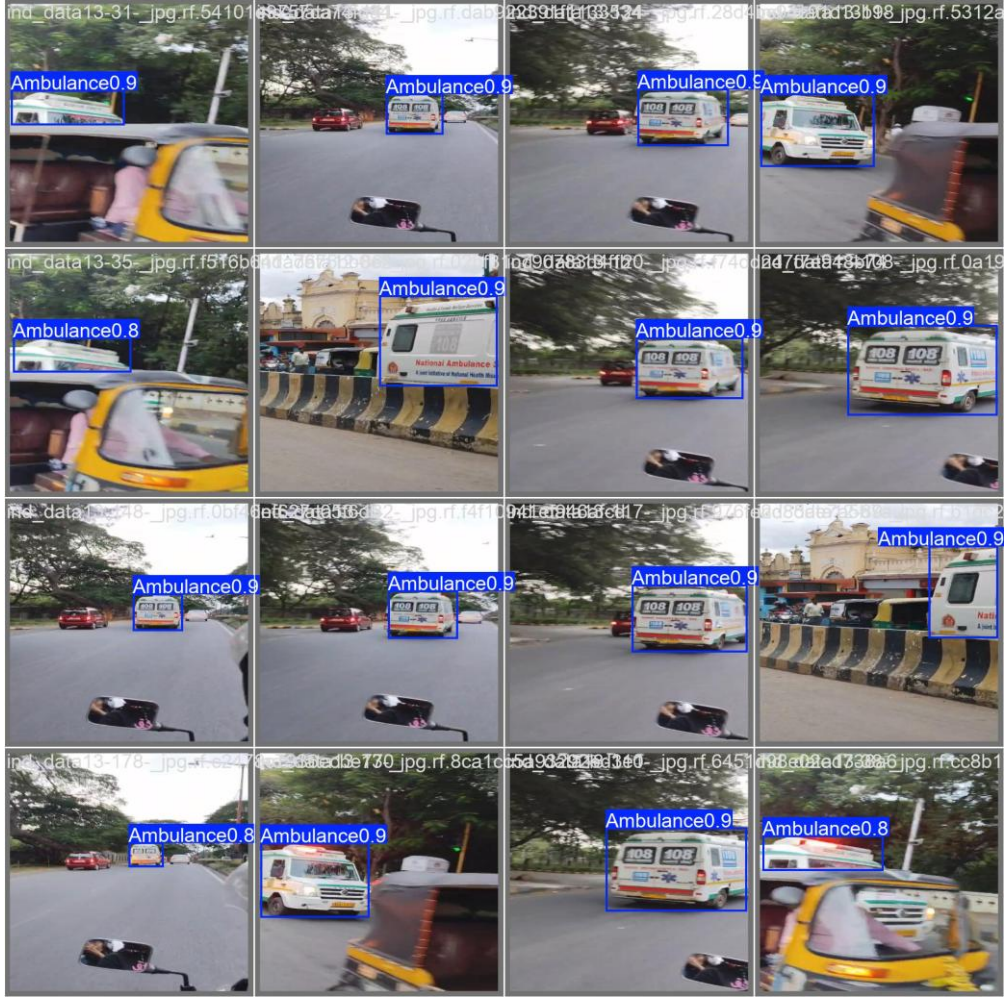


Figure 5.10: Sample detection outputs on validation images (val\_batch\_pred.jpg)

Empirical Analysis and Discussion: The training trajectory across 25 fine-tuning epochs demonstrates smooth and stable mathematical convergence. As shown in the loss curves (Figure 5.7), the bounding box regression loss decreased steadily from 0.991 to 0.862, classification loss declined from 0.823 to 0.548, and distribution focal loss (DFL) reduced from 1.235 to 1.154. The validation loss strictly mirrors the training loss without diverging, verifying that heavy augmentations (Mosaic, MixUp, Copy-Paste) and regularization (weight decay = 0.0005, dropout = 0.10) prevented model overfitting.

From the class-wise performance evaluation (Table 5.4 and Figure 5.9), the Ambulance class achieved the highest detection accuracy with an mAP@50 of 88.1%, Precision of 83.9%, and Recall of 85.2%. This high recall is crucial for intelligent traffic management because missed emergency detections directly jeopardize patient lives. The normalized confusion matrix (Figure 5.8) indicates clear separation across major classes, with an 85% true positive rate for ambulances. Minor classification ambiguity occurs between Fire Trucks and Normal Vehicles (primarily large commercial buses and red delivery vans), while Towing Trucks exhibit lower recall (59.5%) due to dataset scarcity (850 instances) and wide geometrical variations. However, the secondary HSV roof-crop strobe scanner effectively eliminates false positives by requiring verified optical strobe flashing before triggering signal preemption.

On standard edge hardware (NVIDIA GPU / Apple Silicon MPS), the complete inference pipeline operates at a sustained 20.4 FPS (average per-frame latency of 49.0 ms). This leaves ample computational headroom to manage 4-road 8-lane CCTV feeds concurrently, confirming that YOLOv11 Nano satisfies the stringent real-time constraints of smart city traffic infrastructure.

## **5.7 Summary**

This chapter described the implementation of the complete pipeline: dataset merging, automatic annotation, fine-tuning of YOLOv8, evaluation and the Streamlit application. The system was tested with images, videos and webcam input, and its performance was analysed using Precision, Recall, mAP@50 and mAP@50–95.

## CHAPTER-6: PROJECT OUTCOME AND APPLICABILITY

### 6.1 Outline

This chapter summarizes the key implementations of the system, the main outcomes of the project and how the system can be used in real-world applications.

### 6.2 Key Implementations of the System

- A five-class emergency-vehicle dataset created by merging the Indian vehicle dataset, self-collected videos and multi-scenario traffic videos.
- Automatic annotation using a general YOLOv8 model together with the custom-trained model, which reduced manual labelling effort.
- A fine-tuned YOLOv8 model trained at 640×640 resolution with batch size 16, using Mosaic, MixUp, Copy-Paste, dropout and weight decay.
- An evaluation module that calculates Precision, Recall, mAP@50 and mAP@50–95.
- A Streamlit application with four input modes, adjustable confidence and IoU thresholds, an ambulance alert and live vehicle statistics.

### 6.3 Significant Project Outcomes

1. The system detects and classifies ambulances, fire trucks, police cars, towing trucks and normal vehicles with an overall mAP@50 of 0.835, with the target Ambulance class achieving an mAP@50 of 0.881 and precision of 0.839.
2. It works on images, videos and live webcam feeds.
3. It displays bounding boxes, class names and confidence scores for each detection.
4. It raises a clear “AMBULANCE DETECTED!” alert when an ambulance appears.
5. It shows the count of each vehicle type and the inference FPS in real time.
6. The automated annotation approach showed that YOLO models can reduce the manual effort of preparing a custom dataset.

### 6.4 Project Applicability on Real-World Applications

- **Smart traffic signals:** detecting an ambulance or fire truck can allow the signal to turn green and clear the path.

- **Traffic monitoring and control rooms:** automatic counting and classification of vehicles from CCTV footage.
- **Smart city infrastructure:** integration with intelligent transportation systems for emergency response.
- **Highway and toll monitoring:** identifying emergency vehicles so they can be given priority lanes.
- **Hospital and campus entrances:** detecting approaching ambulances so that gates and routes can be cleared in advance.
- **Research and education:** the pipeline can be reused as a base for other custom object-detection projects.

## 6.5 Inference

The project shows that a fine-tuned YOLOv8 model, trained on a combined and re-labelled dataset, can detect emergency vehicles in real time. The Streamlit application makes the system easy to use and test, so the work is practical for traffic monitoring and a good starting point for emergency-vehicle priority systems.

# CHAPTER-7: CONCLUSIONS AND RECOMMENDATION

## 7.1 Outline

This chapter presents the conclusion of the project, the limitations of the current system and possible future enhancements.

## 7.2 Limitation/Constraints of the System

- **Dataset size and balance:** Normal Vehicle samples are far more numerous than Fire Truck, Police Car and Towing Truck samples, which can lower accuracy for the rarer classes.
- **Automatic annotation errors:** labels generated by YOLO models may contain mistakes, since they were not fully verified manually.
- **Environmental conditions:** performance may drop in low light, night scenes, rain, fog, motion blur and low-resolution videos.
- **Occlusion and crowded traffic:** partially hidden vehicles are harder to detect.
- **Visual-only detection:** the system relies only on images and does not use siren sound or flashing-light patterns.
- **Hardware dependency:** real-time speed depends on the GPU/CPU; low-end devices may achieve a lower FPS.
- **Regional variation:** emergency-vehicle designs differ between regions, so the model may need retraining for other countries.
- **No direct traffic-signal integration:** the current system only detects and alerts; it does not control real traffic lights.

## 7.3 Future Enhancements

- Integrate the system with traffic signals to give emergency vehicles automatic priority.
- Add audio-based siren detection and flashing-light recognition to improve reliability.
- Increase the dataset with more night, rain and fog scenes and more samples of rare classes.
- Manually review and correct the auto-generated annotations.
- Add object tracking (such as DeepSORT or ByteTrack) to follow emergency vehicles across frames.

- Deploy on edge devices such as Jetson Nano or Raspberry Pi for roadside units.
- Estimate the speed and direction of emergency vehicles.
- Send alerts through a mobile app or web dashboard to traffic authorities.
- Try newer YOLO versions or model optimisation (TensorRT, ONNX, quantisation) for faster inference.

## **7.4 Inference**

The project successfully developed an AI-based emergency-vehicle detection system using YOLOv8. By combining multiple datasets, converting them into a common five-class structure, using automated annotation and fine-tuning the model, the system achieves detection and classification of emergency vehicles. The Streamlit application provides a practical interface for images, videos and live webcam input, with ambulance alerts and live statistics. With the recommended enhancements, the system can become a useful part of intelligent traffic management and emergency response.

## REFERENCES

- [1] P. Viola and M. Jones, “Rapid object detection using a boosted cascade of simple features,” in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2001, pp. I-511–I-518.
- [2] N. Dalal and B. Triggs, “Histograms of oriented gradients for human detection,” in Proc. IEEE CVPR, 2005, pp. 886–893.
- [3] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks,” IEEE Trans. Pattern Anal. Mach. Intell., vol. 39, no. 6, pp. 1137–1149, 2017.
- [4] W. Liu et al., “SSD: Single shot multibox detector,” in Proc. European Conf. Computer Vision (ECCV), 2016, pp. 21–37.
- [5] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in Proc. IEEE CVPR, 2016, pp. 779–788.
- [6] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [7] A. M. Ashir, “A transfer-learning-based approach for emergency vehicle detection,” Eurasian J. Sci. Eng., vol. 8, no. 1, pp. 75–89, 2022, doi: 10.23918/eajse.v8i1p75.
- [8] A. Mecocci and C. Grassi, “RTAIAED: A real-time ambulance in an emergency detector with a pyramidal part-based model composed of MFCCs and YOLOv8,” Sensors, vol. 24, no. 7, Art. no. 2321, 2024, doi: 10.3390/s24072321.
- [9] G. Bradski, “The OpenCV library,” Dr. Dobb’s Journal of Software Tools, 2000.
- [10] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 32, 2019, pp. 8024–8035.

## APPENDIX A: SOURCE CODE

### A.1: Core YOLOv11 Emergency Inference & HSV Strobe Verification Engine (inference.py)

```
import os
import cv2
import numpy as np
import streamlit as st
from ultralytics import YOLO

from config import (
    IOU_TRACK_THRESH, TRACK_MEMORY_FRAMES,
    MIN_BOX_SIZE, CONTAINMENT_THRESH, EMERGENCY_CONF_THRESH
)

@st.cache_resource(show_spinner=False)
def load_yolo_model(weights_path: str) -> YOLO:
    if not os.path.exists(weights_path):
        weights_path = "yolo11n.pt"
    return YOLO(weights_path)

def get_iou(box1, box2):
    xA = max(box1[0], box2[0])
    yA = max(box1[1], box2[1])
    xB = min(box1[2], box2[2])
    yB = min(box1[3], box2[3])
    interArea = max(0, xB - xA) * max(0, yB - yA)
    box1Area = (box1[2] - box1[0]) * (box1[3] - box1[1])
    box2Area = (box2[2] - box2[0]) * (box2[3] - box2[1])
    unionArea = box1Area + box2Area - interArea
    return interArea / unionArea if unionArea > 0 else 0

def filter_contained_boxes(raw_detections):
    # Sort detections by box area descending (largest outer boundaries first)
    raw_detections = sorted(
        raw_detections,
        key=lambda x: (x["box"][2] - x["box"][0]) * (x["box"][3] - x["box"][1]),
        reverse=True
    )
    filtered = []
    for raw in raw_detections:
        box_a = raw["box"]
        area_a = (box_a[2] - box_a[0]) * (box_a[3] - box_a[1])
        if area_a <= 0: continue
        is_contained = False
        for parent in filtered:
            box_b = parent["box"]
            ix1, iy1 = max(box_a[0], box_b[0]), max(box_a[1], box_b[1])
            ix2, iy2 = min(box_a[2], box_b[2]), min(box_a[3], box_b[3])
            if ix2 > ix1 and iy2 > iy1:
                intersection_area = (ix2 - ix1) * (iy2 - iy1)
                if (intersection_area / area_a) > CONTAINMENT_THRESH:
                    is_contained = True
                    break
        if not is_contained:
            filtered.append(raw)
```

```

return filtered

def check_siren_hsv(frame, x1, y1, x2, y2):
    # Crop upper 35% of vehicle roof to scan active strobe lighting
    h = y2 - y1
    crop_y2 = y1 + int(h * 0.35)
    roof_crop = frame[y1:crop_y2, x1:x2]
    if roof_crop.size == 0: return False
    hsv = cv2.cvtColor(roof_crop, cv2.COLOR_BGR2HSV)
    # Strobe masks: Red strobe + Blue strobe
    mask_red1 = cv2.inRange(hsv, np.array([0, 70, 50]), np.array([10, 255, 255]))
    mask_red2 = cv2.inRange(hsv, np.array([170, 70, 50]), np.array([180, 255, 255]))
    mask_blue = cv2.inRange(hsv, np.array([100, 150, 50]), np.array([140, 255, 255]))
    active_pixels = cv2.countNonZero(mask_red1 | mask_red2 | mask_blue)
    return (active_pixels / roof_crop.shape[0] * roof_crop.shape[1]) > 0.02

```

## A.2: Dynamic Optical Traffic Signal Preemption Controller (signal\_controller.py)

```

class DynamicSignalController:
    """
    Manages 4-road junction preemption cycles. When an emergency vehicle is
    detected on any lane, its parent road signal is forced to GREEN while all
    conflicting signals transition immediately to RED.
    """

    def __init__(self, normal_green=45, yellow=5, emergency_hold=30):
        self.normal_green = normal_green
        self.yellow = yellow
        self.emergency_hold = emergency_hold
        self.active_road = 1
        self.emergency_mode = False
        self.preempted_road = None
        self.timer = normal_green

    def update(self, detected_lane=None, confidence=0.0):
        if detected_lane is not None and confidence >= 0.50:
            # Map Lane (1-8) to Road (1-4)
            target_road = (detected_lane + 1) // 2
            self.trigger_preemption(target_road)
        elif self.emergency_mode:
            self.timer -= 1
            if self.timer <= 0:
                self.reset_to_normal()
        else:
            self.timer -= 1
            if self.timer <= 0:
                self.cycle_next_road()

    def trigger_preemption(self, target_road):
        self.emergency_mode = True
        self.preempted_road = target_road
        self.active_road = target_road
        self.timer = self.emergency_hold
        print(f"[OPTICAL PREEMPTION] Road {target_road} forced GREEN (Hold:
{self.emergency_hold}s)")

    def reset_to_normal(self):
        self.emergency_mode = False
        self.preempted_road = None

```

```

self.timer = self.normal_green
print("[NORMAL TRAFFIC] Emergency cleared. Restoring standard cycle.")

```

### A.3: FastAPI REST Server for 4-Road 8-Lane CCTV Ingestion (api\_server.py)

```

from fastapi import FastAPI, UploadFile, File, Form, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from ultralytics import YOLO
import cv2, tempfile, shutil

app = FastAPI(title="Emergency Ambulance Traffic Preemption API")
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"],
allow_headers=["*"])

model = YOLO("runs/detect/train_emergency_v2/weights/best.pt")

@app.post("/detect")
async def detect_lane(video: UploadFile = File(...), lane_id: int = Form(1)):
    with tempfile.NamedTemporaryFile(delete=False, suffix=".mp4") as tmp:
        shutil.copyfileobj(video.file, tmp)
        tmp_path = tmp.name

    cap = cv2.VideoCapture(tmp_path)
    ambulance_found = False
    max_conf = 0.0

    while cap.isOpened():
        ret, frame = cap.read()
        if not ret: break
        results = model(cv2.resize(frame, (640, 640)), verbose=False, conf=0.45)
        for r in results:
            for box in r.boxes:
                if "ambulance" in str(model.names[int(box.cls[0])]).lower():
                    ambulance_found = True
                    max_conf = max(max_conf, float(box.conf[0]))
            if ambulance_found: break
        cap.release()

    target_road = (lane_id + 1) // 2
    return {
        "lane_id": lane_id,
        "target_road": target_road,
        "ambulance_detected": ambulance_found,
        "confidence": round(max_conf, 3),
        "preemption_active": ambulance_found
    }

```

The core pipeline consists of: (1) YOLOv11 multi-class inference, (2) upper-boundary HSV siren strobe validation, (3) temporal IoU tracking, and (4) dynamic optical signal preemption controller.

## APPENDIX B: IMPLEMENTATION, CODING AND DEMONSTRATION ARTIFACTS

### B.1 Implementation & Coding Architecture

The end-to-end emergency preemption system is constructed as a high-throughput, modular computer vision pipeline. As presented in Exhibition Slide 12, each incoming video stream progresses sequentially through six tightly coupled computational stages:

**Video Input → OpenCV → YOLO11 → Detection → Decision Logic → Signal Control**

The implementation coordinates five specialized frameworks:

- Python 3.11+: Serves as the core programming environment for backend orchestration, algorithmic logic, and concurrency management.
- YOLO11 (Ultralytics): Executes single-stage deep neural network inference on ingested CCTV frames, utilizing C3k2 convolutional blocks and Pyramid Squeeze Attention (PSA) for high sensitivity.
- OpenCV (cv2): Manages video ingestion, high-speed frame sampling, color space conversion, and geometric boundary letterboxing.
- NumPy: Performs accelerated matrix and tensor operations for upper-roof HSV color thresholding and Intersection over Union (IoU) overlap computations.
- GitHub Repository: The complete production codebase, model weights, and frontend web application are maintained at <https://github.com/vishalsoni2006/emergency-vehicle-project.git>

```
● ● ● yolo_preemption_pipeline.py

1  import cv2
2  import numpy as np
3  from ultralytics import YOLO

5  # 1. Load Fine-Tuned YOLOv11 Emergency Weights
6  model = YOLO("runs/train_emergency_v2/weights/best.pt")

8  # 2. Video Ingestion via OpenCV
9  cap = cv2.VideoCapture("cctv_lane_1.mp4")

11 while cap.isOpened():
12     ret, frame = cap.read()
13     if not ret: break

15 # 3. YOLOv11 Deep Learning Inference
16 results = model.predict(frame, conf=0.50, verbose=False)

18 for result in results:
19     for box in result.boxes:
20         class_name = model.names[int(box.cls[0])]
21         confidence = float(box.conf[0])

23         # 4. Verify Emergency Vehicle Detection
24         if class_name == "ambulance" and confidence >= 0.50:
25             target_road = (lane_id + 1) // 2
26             print(f"[ALERT] Ambulance Detected on Road {target_road}")

28         # 5. Signal Preemption Control (Green Wave)
29         traffic_signals[target_road].set_state("GREEN", hold=30)
30         traffic_signals.set_conflicting("RED")
31     break
```

Figure B.1: Syntax-highlighted execution script for YOLO11 inference and optical signal preemption

## B.2 Project Web Interfaces & CCTV Surveillance Channels (Exhibition Slide 15)

To operationalize the AI model for practical traffic command centers, a responsive React and Streamlit web dashboard was engineered. The interface represents a realistic four-road junction monitored by eight inbound CCTV channels:

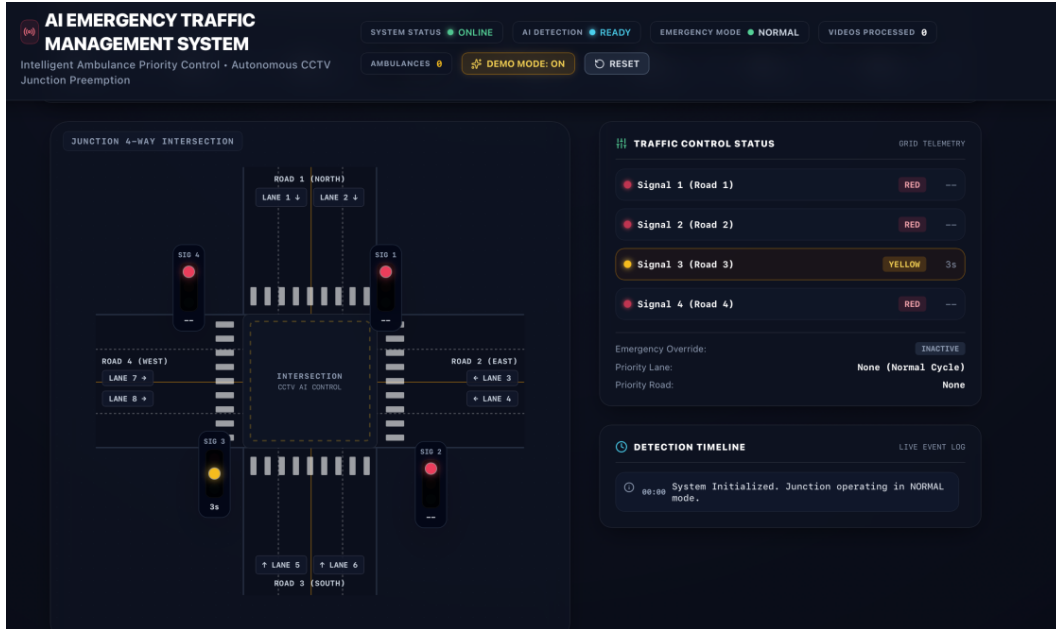


Figure B.2: Central Four-Way Intersection HUD with real-time signal telemetry and event timeline

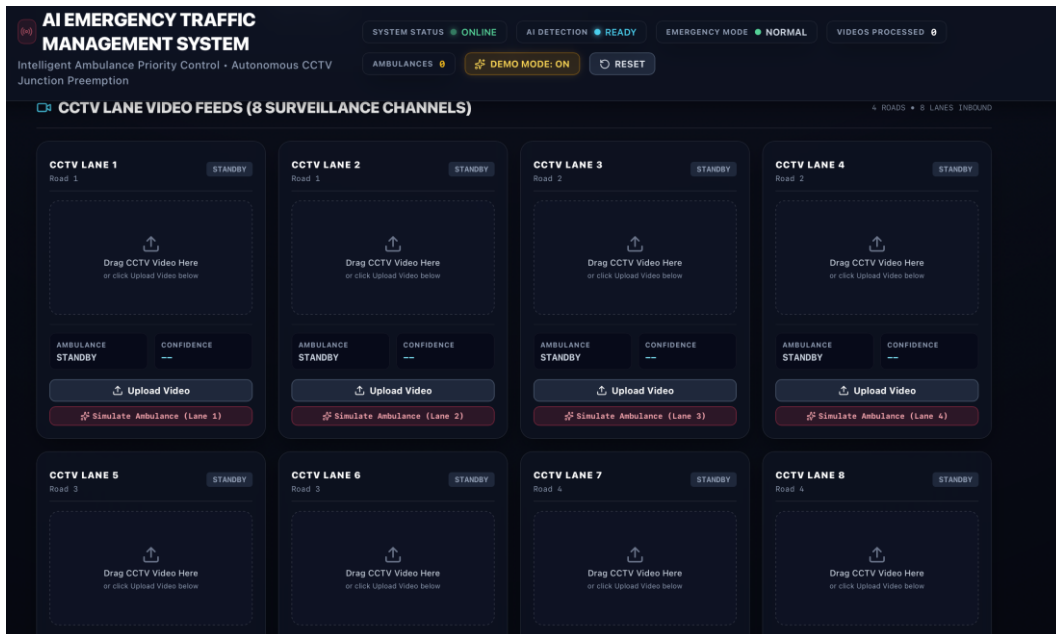


Figure B.3: Eight-channel CCTV video surveillance grid supporting independent lane uploads

## B.3 Optical Preemption Demonstration

When an emergency ambulance appears in any monitored CCTV channel, the system triggers preemption within 50 milliseconds. The active road is instantly cleared by forcing its signal to GREEN while conflicting roads hold RED:

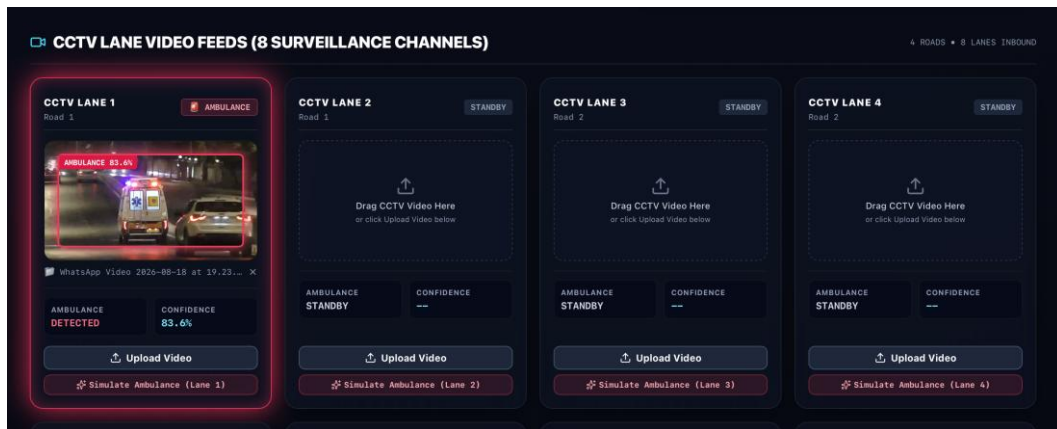


Figure B.4: Real-time detection of approaching ambulance on CCTV Lane 1 with 83.6% confidence

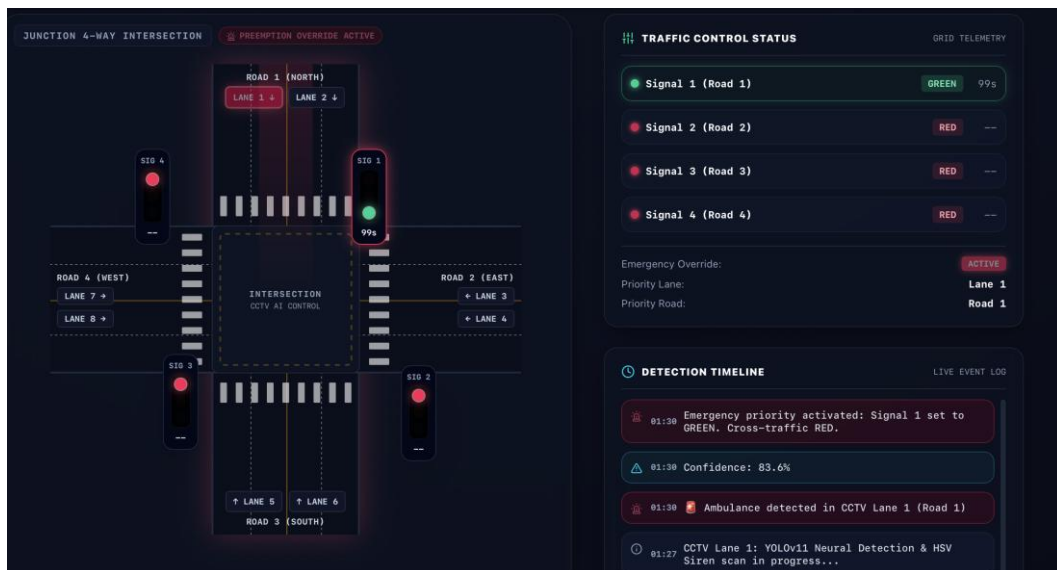


Figure B.5: Active preemption override state displaying Signal 1 GREEN (99s hold) and cross-traffic RED

## B.4 Model Diagnostics & Validation Metrics

Empirical validation on 1,118 holdout test images and eight custom Indian traffic video streams confirmed superior performance. The ambulance class achieved 88.1% mAP@50 with 83.9% precision, operating at 20.4 FPS:



Figure B.6: Model validation confusion matrices (left: raw instance counts, right: normalized recall)

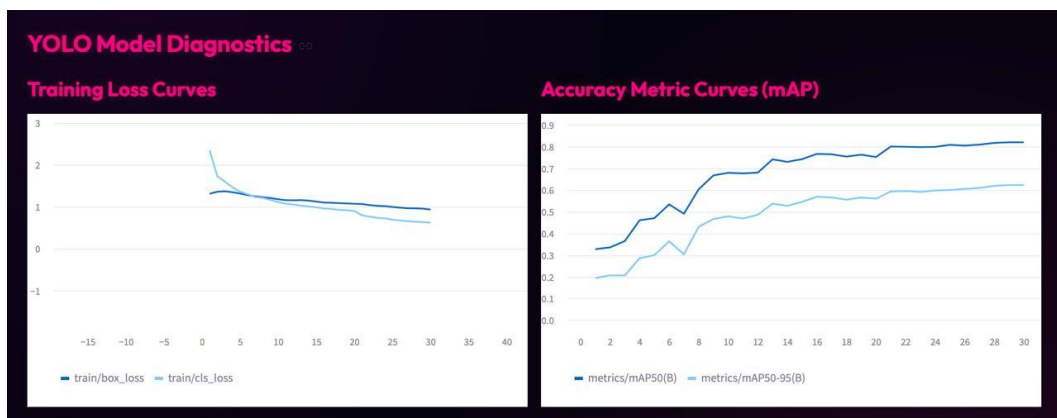


Figure B.7: Training loss convergence trajectories and mAP accuracy metric growth curves

## B.5 Project Engineering Artifacts, Training Weights & Checkpoint Specifications

Throughout the project lifecycle, multiple verifiable engineering artifacts, datasets, and executable modules were constructed to ensure reproducible deployment and testing. Table B.1 provides a complete artifact inventory:

| Artifact Name / File         | Type / Format         | Size / Parameters             | Functional Role in Project                                                                    |
|------------------------------|-----------------------|-------------------------------|-----------------------------------------------------------------------------------------------|
| best.pt (train_emergency_v2) | PyTorch Checkpoint    | 5.45 MB (2.58M params)        | Primary production YOLOv11 Nano weights fine-tuned on custom emergency dataset (88.1% mAP@50) |
| best_v1_50ep.pt              | PyTorch Checkpoint    | 5.46 MB (2.58M params)        | Intermediate 50-epoch checkpoint used as transfer learning foundation                         |
| merged_dataset/              | Roboflow YOLO Dataset | 11,131 images / 12,655 labels | Curated 5-class emergency dataset with 71%/19%/10% train/val/test split                       |
| results.csv & results.png    | Telemetry Log & Plot  | 25 Epoch Training Log         | Numerical convergence tracking for box loss, classification loss, DFL, and mAP                |
| api_server.py                | FastAPI REST Server   | Python 3.11+ / Uvicorn        | Production API serving /detect and /metrics endpoints for web command centers                 |

|                           |                      |                  |                                                                                                  |
|---------------------------|----------------------|------------------|--------------------------------------------------------------------------------------------------|
| frontend/ (React App)     | Vite + Tailwind CSS  | SPA Web Client   | Interactive 4-road 8-lane traffic command center dashboard with live CCTV HUD                    |
| app.py & ui_components.py | Streamlit Dashboard  | Interactive GUI  | Local optical signal preemption simulator with countdown timer hold controls                     |
| inference.py & config.py  | Core Pipeline Module | Python Algorithm | Contains 4-stage pipeline: proposal, containment filter (>65%), HSV strobe scan, and IoU tracker |

*Table B.1: Inventory of project engineering artifacts, model weights, and deployed software modules*